

Reviewer Pack — Monomial Count in KW Polynomial Bounds Communication Complex...

Entry 0c5560c32966 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 20:59:48 UTC

Monomial Count in KW Polynomial Bounds Communication Complexity

Entry ID: 0c5560c32966 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 20:59:48 UTC

1. Conjecture

Field A (mathematical branch): Polynomial Idempotent Decomposition **Field B** (complexity object): Karchmer-Wigderson Communication Complexity

Statement:

For any Boolean function $f: \{0,1\}^{n \times \{0,1\}} \rightarrow \{0,1\}$, the number of monomials in the minimal idempotent decomposition of its KW communication matrix over $\text{GF}(2)$ is $\Theta(2^{\{n/2\}})$

Rationale (proposer’s reasoning):

Idempotent decompositions reveal algebraic structure in communication matrices, while KW complexity captures formula size. Their connection may expose hidden symmetries in function representation

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc7ef3c176b83239

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot row
        max_row = i
        for r in range(i+1, rows):
            if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                max_row = r
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate non-pivot elements
        for j in range(cols):
            if i != j:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(rows):
                    matrix[j][k] -= factor * matrix[i][k]

def is_idempotent(matrix):
    return gaussian_elimination(matrix) == 0

def count_monomials(idempotents):
    return sum(1 for idempotent in idempotents if is_idempotent(idempotent))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    instances_tested = 30
    total_monomial_count = 0

    for _ in range(instances_tested):
        # Generate a random 2-CNF formula with n variables
        clauses = []
        for _ in range(n * (n - 1) // 2):
            literals = [random.choice([f'x{i+1}', f'x{i+1}']) for i in range(n)]
            clause = ' or '.join(literals)
            clauses.append(clause)

        # Construct the communication matrix
        comm_matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            for j in range(i, n + 1):
                if random.choice([True, False]):
```

```

        comm_matrix[i][j] = 1
        comm_matrix[j][i] = 1

    # Decompose into idempotents (simplified version)
    idempotents = [comm_matrix]

    monomial_count = count_monomials(idempotents)
    total_monomial_count += monomial_count

mean_monomial_count = total_monomial_count / instances_tested
conjecture_holds = abs(mean_monomial_count - 2**(n/2)) < 1e-6
counterexample = "monomial_count_deviation" if not conjecture_holds else ""

    return {
        "metric_name": "monomial_count",
        "metric_value": mean_monomial_count,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [727, 773, 821, 877, 929] # Default to a list of primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_monomial_count = sum(r['metric_value'] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_monomial_count} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_monomial_count} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8e88ffd.py", line 87, in <module>
    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8e88ffd.py", line 66, in run_trial
    monomial_count = count_monomials(idempotents)

```


12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0c5560c32966.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0c5560c32966.tar.gz` (if generated)